

EJEMPLAR DE LA OBRA — CÓDIGO FUENTE (EXTRACTO)

Plataforma Digital de Pedidos

Autor y titular: Jose Miguel Gonzalez Domingo

Extracto representativo: primeras 1000 y últimas 1000 líneas del listado de código fuente

(Código fuente total: 130 ficheros, 14257 líneas)

```

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/ClientApp.tsx */
/* ===== */
import { useState, useCallbck, useEffect, useRef } from 'react';
import { useSearchParams } from 'react-router-dom';
import { X } from 'lucide-react';
import { ClientHeader } from '@components/client/ClientHeader';
import { LocalSelector } from '@components/client/LocalSelector';
import { useActiveLocal } from '@lib/active-local';
import { CategoryGrid } from '@components/client/CategoryGrid';
import { SectionTabs } from '@components/client/SectionTabs';
import { ProductCard } from '@components/client/ProductCard';
import { MontyRuedaCard } from '@components/client/MontyRuedaCard';
import { CartSheet } from '@components/client/CartSheet';
import { AlcoholWarningSheet } from '@components/client/AlcoholWarningSheet';
import { AllergenFilterSheet } from '@components/client/AllergenFilterSheet';
import { AllergenLegalNoticeFloat } from '@components/client/AllergenLegalNotice';
import { OrderTracking } from '@components/client/OrderTracking';
import { MenuChatBot } from '@components/client/MenuChatBot';
import { WelcomeScreen } from '@components/client/WelcomeScreen';
import { MontyLoader } from '@components/client/MontyLoader';
import { useCategories, useProducts } from '@hooks/use-menu';
import { useAllergens } from '@hooks/use-allergens';
import { useLocalRestricciones, buildRestriccionFilter } from '@hooks/use-local-restricciones';
import { useAgotados } from '@hooks/use-agotados';
import { useCartStore } from '@lib/cart-store';
import { useAllergenFilter } from '@lib/allergen-filter';
import { supabase } from '@integrations/supabase/client';
import { EmbeddedCheckout } from '@components/client/EmbeddedCheckout';
import { toast } from 'sonner';
import { registerPushServiceWorker, usePushSubscription } from '@hooks/use-push-notifications';

const WELCOME_KEY = 'monty-welcome-seen';
const ACTIVE_ORDER_KEY = 'monty-active-order';
const BEBIDAS_CATEGORY_NAME = 'Bebidas';
const INITIAL_STAFF_STATUS = 'pendiente_pago' as const;

const MONTADITO_SECTIONS = [
  'De la casa',
  'Clásicos',
  'Imprescindibles',
  'Especiales',
  'MontyCookie',
  'MontyDinas',
  'MontyPerros',
  'MontyBurgers',
  'MontyPizzas',
  'MontyGourmet',
] as const;

const BEBIDAS_SECTIONS = [
  // Jarras y cervezas arriba del todo
  'Jarras Heladas',
  'Cerveza Premium',
  'Cerveza en Botella',
  // Luego las bebidas clásicas
  'Clásicas',
  // Resto
  'Energéticas',
  'Tardeo Chill',
  'Tardeo Premium',
  'Vino',
  'Café e Infusiones',
] as const;

const APERITIVOS_SECTIONS = [
  'Aperitivos',
  'Para picar',
] as const;

type ActiveOrder = {
  pedidoId: string;
  numeroPedido: number;
  hasCocina: boolean;
  hasBebidas: boolean;
  sessionId?: string;
  total?: number;
};

const loadActiveOrder = (): ActiveOrder | null => {
  try {
    const raw = localStorage.getItem(ACTIVE_ORDER_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw);
    // Only pedidoId is required - numeroPedido can be 0 when the DB trigger hasn't fired yet
    if (parsed?.pedidoId) {
      return {
        pedidoId: parsed.pedidoId,
        numeroPedido: parsed.numeroPedido ?? 0,
        hasCocina: parsed.hasCocina ?? true,
      };
    }
  }

```

```

        hasBebidas: parsed.hasBebidas ?? false,
        sessionId: parsed.sessionId,
        total: parsed.total,
    };
}
if (parsed?.cocina || parsed?.bebidas) {
    const ref = parsed.cocina || parsed.bebidas;
    return {
        pedidoId: ref.pedidoId,
        numeroPedido: ref.numeroPedido,
        hasCocina: !!parsed.cocina,
        hasBebidas: !!parsed.bebidas,
        sessionId: ref.sessionId,
    };
}
return null;
} catch {
    return null;
}
};

const ClientApp = () => {
    const [searchParams] = useSearchParams();

    // Escape hatch: ?reset=1 limpia localStorage y recarga.
    // Útil cuando el app queda bloqueado por un pedido corrupto en storage.
    if (searchParams.get('reset') === '1') {
        localStorage.removeItem(ACTIVE_ORDER_KEY);
        window.location.replace(window.location.pathname);
        return null;
    }

    const localSlugFromUrl = searchParams.get('local');
    const mesaNum = searchParams.get('mesa');

    const activeLocal = useActiveLocal((s) => s.local);
    const setActiveLocal = useActiveLocal((s) => s.setLocal);

    const [activeCategory, setActiveCategory] = useState<string | null>(null);
    const [activeSection, setActiveSection] = useState<string | null>(null);
    const [cartOpen, setCartOpen] = useState(false);
    const [welcomeOpen, setWelcomeOpen] = useState(false);
    const [showLocalSelector, setShowLocalSelector] = useState(false);
    const [orderState, setOrderState] = useState<ActiveOrder | null>(() => {
        // Recuperación sincrónica en el primer render para evitar flash de LocalSelector
        // cuando iOS borra localStorage tras el redirect cross-domain de Apple Pay.
        const urlParams = new URLSearchParams(window.location.search);
        if (urlParams.get('pago') !== 'ok') return null;
        const saved = loadActiveOrder();
        if (saved) return saved;
        const pid = urlParams.get('pid');
        const num = urlParams.get('num');
        const sid = urlParams.get('sid');
        if (!pid || num === null) return null;
        const recovered: ActiveOrder = {
            pedidoId: pid,
            numeroPedido: parseInt(num, 10),
            hasCocina: urlParams.get('cocina') !== '0',
            hasBebidas: urlParams.get('bebidas') === '1',
            sessionId: sid ?? undefined,
        };
    });
    localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(recovered));
    return recovered;
});
const [pendingCheckout, setPendingCheckout] = useState<ActiveOrder | null>(null);
const [resumableOrder, setResumableOrder] = useState<ActiveOrder | null>(null);
// Si venimos del redirect de Apple Pay (?pago=ok), empezamos sin recovering
// para que MontyLoader NUNCA bloquee la pantalla. El efecto pago=ok restaura el pedido.
const [recovering, setRecovering] = useState(() => {
    const urlParams = new URLSearchParams(window.location.search);
    if (urlParams.get('pago') === 'ok') return false;
    return !!loadActiveOrder()?.pedidoId;
});
const [isCheckedOut, setIsCheckingOut] = useState(false);
const [filterOpen, setFilterOpen] = useState(false);
const { data: allAllergens = [] } = useAllergens();
const excluded = useAllergenFilter((s) => s.excluded);
const toggleExcluded = useAllergenFilter((s) => s.toggle);

// Refs para el handler de back button (evita closures obsoletos)
const pendingCheckoutRef = useRef<ActiveOrder | null>(pendingCheckout);
const activeCategoryRef = useRef<string | null>(activeCategory);
const activeSectionRef = useRef<string | null>(activeSection);
const handlePaymentCancelRef = useRef<() => void>(() => {});
useEffect(() => { pendingCheckoutRef.current = pendingCheckout; }, [pendingCheckout]);
useEffect(() => { activeCategoryRef.current = activeCategory; }, [activeCategory]);
useEffect(() => { activeSectionRef.current = activeSection; }, [activeSection]);

// Botón atrás del móvil – mapea a navegación interna
// Siempre hay una entrada en el historial para interceptar el back.
// Después de cada navegación se re-inserta la guardia para que el
// siguiente back también sea interceptado (sin esto, solo funciona una vez).

```

```

useEffect(() => {
  window.history.pushState({ step: 'root' }, '');

  const onPopState = () => {
    if (pendingCheckoutRef.current) {
      // Opción A: volver al menú con el pedido recuperable, sin cancelar
      setResumableOrder(pendingCheckoutRef.current);
      setPendingCheckout(null);
      setActiveCategory(null);
      setActiveSection(null);
      window.history.pushState({ step: 'root' }, '');
    } else if (activeSectionRef.current) {
      setActiveSection(null);
      window.history.pushState({ step: 'category' }, '');
    } else if (activeCategoryRef.current) {
      setActiveCategory(null);
      window.history.pushState({ step: 'root' }, '');
    } else {
      window.history.pushState({ step: 'root' }, '');
    }
  };

  window.addEventListener('popstate', onPopState);
  return () => window.removeEventListener('popstate', onPopState);
}, []);

useEffect(() => { registerPushServiceWorker(); }, []);

// Recuperar sesión al arrancar: verificar en DB el estado real del pedido guardado
useEffect(() => {
  const saved = loadActiveOrder();
  if (!saved?.pedidoId) { setRecovering(false); return; }

  // Venimos del redirect de Apple Pay / Google Pay (?pago=ok).
  // Llamamos confirm-payment para que el pedido salga de pendiente_pago inmediatamente
  // sin depender del timing del webhook de Stripe.
  const params = new URLSearchParams(window.location.search);
  if (params.get('pago') === 'ok') {
    const paymentIntentId = params.get('payment_intent');
    if (paymentIntentId && saved.pedidoId && saved.sessionId) {
      supabase.functions.invoke('confirm-payment', {
        body: { pedido_id: saved.pedidoId, session_id: saved.sessionId, payment_intent_id: paymentIntentId },
      }).catch((e) => console.error('[confirm-payment redirect]', e));
    }
    setOrderState(saved);
    setRecovering(false);
    return;
  }

  // Timeout de seguridad: si la query DB tarda más de 5 s, mostramos el
  // pedido igualmente. Evita la pantalla beige infinita por red lenta.
  const safetyTimer = window.setTimeout(() => {
    setOrderState(saved);
    setRecovering(false);
  }, 5000);

  (async () => {
    try {
      // RLS exige la cabecera de sesión: sin ella la lectura devuelve 0 filas
      // y el pedido (¡ya pagado!) se interpretaba como inexistente y se borraba.
      const { data: pedido, error } = await supabase
        .from('pedidos')
        .select('estado')
        .eq('id', saved.pedidoId)
        .setHeader('x-session-id', saved.sessionId ?? '')
        .maybeSingle();
      // Regla de oro: NUNCA borrar el pedido local ante una lectura ambigua
      // (error de red / RLS). Solo se descarta ante un estado terminal explícito.
      if (error || !pedido) {
        setOrderState(saved);
      } else if (pedido.estado === 'cancelado' || pedido.estado === 'entregado') {
        localStorage.removeItem(ACTIVE_ORDER_KEY);
      } else if (pedido.estado === 'pendiente_pago') {
        // Con activeLocal null (iOS borró localStorage tras redirect Apple Pay),
        // ir directo a OrderTracking evita quedarse bloqueado en LocalSelector.
        // OrderTracking muestra "Pago pendiente" + botón "Reintentar pago".
        if (!activeLocal) {
          setOrderState(saved);
        } else {
          setResumableOrder(saved);
        }
      } else {
        setOrderState(saved);
      }
    } catch {
      // Sin red: mostrar seguimiento (OrderTracking se autocorriga al reconectar)
      setOrderState(saved);
    } finally {
      window.clearTimeout(safetyTimer);
      setRecovering(false);
    }
  })();

```

```

// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// If a `local` slug is provided in the URL (e.g. QR), bind that local automatically.
useEffect(() => {
  if (!localSlugFromUrl) return;
  if (activeLocal) return;
  (async () => {
    const { data } = await supabase
      .from('locales')
      .select('id, nombre, ciudad, direccion')
      .eq('slug', localSlugFromUrl)
      .eq('activo', true)
      .maybeSingle();
    if (data) setActiveLocal(data as any);
  })();
}, [localSlugFromUrl, activeLocal, setActiveLocal]);

useEffect(() => {
  if (!activeLocal) return;
  if (!localStorage.getItem(WELCOME_KEY)) {
    setWelcomeOpen(true);
  }
}, [activeLocal]);

// Al volver de Stripe (Apple Pay redirect), mostrar feedback y restaurar el pedido.
useEffect(() => {
  const pago = searchParams.get('pago');
  if (!pago) return;
  if (pago === 'ok') {
    toast.success('¡Pago realizado con éxito!');

    // Stripe añade payment_intent al return_url en redirects (Apple Pay, Google Pay).
    // Lo usamos para llamar confirm-payment inmediatamente y sacar el pedido de
    // pendiente_pago sin esperar al webhook – esto soluciona la pantalla en blanco.
    const paymentIntentId = searchParams.get('payment_intent');
    const pidFromUrl = searchParams.get('pid');
    const sidFromUrl = searchParams.get('sid');
    if (paymentIntentId && pidFromUrl && sidFromUrl) {
      supabase.functions.invoke('confirm-payment', {
        body: { pedido_id: pidFromUrl, session_id: sidFromUrl, payment_intent_id: paymentIntentId },
      }).catch((e) => console.error('[confirm-payment redirect]', e));
    }

    const saved = loadActiveOrder();
    if (saved) {
      setOrderState(saved);
    } else {
      // Fallback: iOS Safari a veces borra localStorage durante el redirect cross-domain.
      // Recuperamos desde los params que EmbeddedCheckout codificó en el return_url.
      const numFromUrl = searchParams.get('num');
      const cocinaFromUrl = searchParams.get('cocina');
      const bebidasFromUrl = searchParams.get('bebidas');
      if (pidFromUrl && numFromUrl !== null) {
        const recovered: ActiveOrder = {
          pedidoId: pidFromUrl,
          numeroPedido: parseInt(numFromUrl, 10),
          hasCocina: cocinaFromUrl !== '0',
          hasBebidas: bebidasFromUrl === '1',
          sessionId: sidFromUrl ?? undefined,
        };
        localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(recovered));
        setOrderState(recovered);
      }
    }
  } else if (pago === 'cancel') {
    toast.error('Pago cancelado');
    localStorage.removeItem(ACTIVE_ORDER_KEY);
  }
  // limpiar query params
  window.history.replaceState({}, '', window.location.pathname);
}, [searchParams]);

const closeWelcome = () => {
  localStorage.setItem(WELCOME_KEY, '1');
  setWelcomeOpen(false);
};

const { subscribe } = usePushSubscription();

const { data: categories = [], isLoading: loadingCategories } = useCategories();
const { data: rawProducts = [] } = useProducts(activeCategory || undefined);
// Montaditos (para resolver los componentes de las MontyRuedas). Misma queryKey
// que cuando la categoría activa es Montaditos → react-query lo deduplica.
const montaditosCatId = (categories as any[]).find((c) => c.nombre === 'Montaditos')?.id;
const { data: allMontaditos = [] } = useProducts(montaditosCatId);
const ruedaMontaditos = (allMontaditos as any[]).map((p) => ({ id: p.id, numero: p.numero ?? null, nombre: p.nombre }));
const { data: restricciones = [] } = useLocalRestricciones();
const { isProductoAllowed, isSeccionAllowed: isSeccionAllowedDB } = buildRestriccionFilter(restricciones);
const { isProductoAgotado, isIngredienteAgotado } = useAgotados();

const now = new Date();

```

```

const isAfter17 = now.getHours() >= 17;
const CAFE_SECTION = 'Café e Infusiones';

// After 17:00: keep Café e Infusiones visible (DB hides it entirely, we override to show only Agua)
const isSeccionAllowed = (sec: string) => {
  if (isAfter17 && sec === CAFE_SECTION) return true;
  return isSeccionAllowedDB(sec);
};

const products = (rawProducts as any[]).filter((p) => {
  if (!isProductoAllowed(p.id)) return false;
  // After 17:00 en Café e Infusiones: solo Agua y los 3 zumos en botella (B41-B43)
  if (isAfter17 && p.seccion === CAFE_SECTION) {
    const nombre: string = (p.nombre ?? '').normalize('NFD').replace(/[~]/g, '').toLowerCase();
    return nombre.includes('agua') || ['B41', 'B42', 'B43'].includes(p.numero);
  }
  return true;
});
const cart = useCartStore();

const selectedCategory = categories.find((c: any) => c.id === activeCategory);

const showLoader = loadingCategories && !welcomeOpen && !activeLocal;

const handleCheckout = useCallback(async (ageVerified: boolean) => {
  if (cart.items.length === 0) return;
  if (isCheckingOut) return;
  if (!activeLocal) {
    toast.error('Selecciona un local primero');
    setShowLocalSelector(true);
    return;
  }
  setIsCheckingOut(true);
  try {
    const localId = activeLocal.id;
    const sessionId = cart.sessionId?.trim();
    if (!sessionId) throw new Error('Sesión de pedido no disponible');
    // Fetch categories for items in cart to split Bebidas vs cocina with the exact shared rule.
    const productIds = Array.from(new Set(cart.items.map((i) => i.productoId)));
    const { data: prodCats, error: prodErr } = await supabase
      .from('menu_productos')
      .select('id, nombre, categoria_id, menu_categorias(nombre)')
      .in('id', productIds);
    if (prodErr) throw prodErr;

    // Normalize a string: remove diacritics, lowercase, trim
    const norm = (s: string) =>
      s.normalize('NFD').replace(/[~]/g, '').toLowerCase().trim();

    // Aperitivos servidos en barra – matched by substring to be resilient to
    // exact DB name variants (e.g. "Gilda boqueron" / "Gilda anchoa" / "Gildas")
    const isAperitivoBarra = (nombre: string) => {
      const n = norm(nombre);
      return n.includes('aceituna') || n.includes('gilda') || n.includes('cucurucho');
    };

    const isBebida = (productoId: string) => {
      const p = prodCats?.find((x) => x.id === productoId);
      if ((p?.menu_categorias as any)?.nombre === BEBIDAS_CATEGORY_NAME) return true;
      if (p?.nombre && isAperitivoBarra(p.nombre)) return true;
      return false;
    };

    const hasBebidas = cart.items.some((i) => isBebida(i.productoId));
    const hasCocina = cart.items.some((i) => !isBebida(i.productoId));

    // Determinar estados iniciales según las partes del pedido
    const total = cart.items.reduce((s, i) => s + i.precio * i.cantidad, 0);
    const { data: pedido, error: pErr } = await supabase
      .from('pedidos')
      .insert({
        local_id: localId,
        numero_pedido: 0,
        total,
        session_id: sessionId,
        estado: INITIAL_STAFF_STATUS,
        tipo: hasCocina && hasBebidas ? 'mixto' : (hasBebidas ? 'bebidas' : 'cocina'),
        estado_cocina: hasCocina ? INITIAL_STAFF_STATUS : null,
        estado_bebidas: hasBebidas ? INITIAL_STAFF_STATUS : null,
        edad_verificada_cliente: ageVerified,
      } as any)
      .select()
      .setHeader('x-session-id', sessionId)
      .single();
    if (pErr) throw pErr;

    const itemRows = cart.items.map((item) => ({
      pedido_id: pedido.id,
      producto_id: item.productoId,
      cantidad: item.cantidad,
    }

```

```

        precio_unitario: item.precio,
        destino: isBebida(item.productoId) ? 'bebidas' : 'cocina',
        // Tamaño/variante elegido (ej. "Jarra Quijote" / "Jarra Sancho") o, en las
        // MontyRuedas, los montaditos que la componen (uno por línea) para cocina.
        variante: (item.componentes && item.componentes.length
            ? item.componentes.join('\n')
            : item.variantLabel) ?? null,
    }));

const { error: iErr } = await supabase
    .from('pedido_items')
    .insert(itemRows as any)
    .setHeader('x-session-id', sessionId);
if (iErr) throw iErr;

const newOrder: ActiveOrder = {
    pedidoId: pedido.id,
    numeroPedido: pedido.numero_pedido,
    hasCocina,
    hasBebidas,
    sessionId,
    total,
};

// Persistimos para soportar recarga durante el pago
localStorage.setItem(ACTIVE_ORDER_KEY, JSON.stringify(newOrder));
cart.clearCart();
setCartOpen(false);
setPendingCheckout(newOrder);
unsubscribe(sessionId, pedido.id).catch(() => {});

} catch (err) {
    console.error('[checkout error]', err);
    const detail =
        err instanceof Error ? err.message : JSON.stringify(err);
    toast.error('Error al crear el pedido: ${detail}');
} finally {
    setIsCheckingOut(false);
}
}, [cart, activeLocal, isCheckingOut]);

const handlePaymentSuccess = useCallback(() => {
    if (!pendingCheckout) return;
    setOrderState(pendingCheckout);
    setPendingCheckout(null);
}, [pendingCheckout]);

const handlePaymentCancel = useCallback(async () => {
    if (pendingCheckout?.pedidoId && pendingCheckout?.sessionId) {
        try {
            await supabase.rpc('cancel_own_pending_order', { _pedido_id: pendingCheckout.pedidoId } as any)
                .setHeader('x-session-id', pendingCheckout.sessionId);
        } catch (e) { console.error(e); }
    }
    localStorage.removeItem(ACTIVE_ORDER_KEY);
    setPendingCheckout(null);
}, [pendingCheckout]);

// Mantener ref siempre actualizado con la versión más reciente
useEffect(() => { handlePaymentCancelRef.current = handlePaymentCancel; });

const handlePaymentBackToMenu = useCallback(() => {
    if (pendingCheckout) {
        setResumableOrder(pendingCheckout);
        setPendingCheckout(null);
    }
}, [pendingCheckout]);

const handleCancelResumable = useCallback(async () => {
    if (resumableOrder?.pedidoId && resumableOrder?.sessionId) {
        try {
            await supabase.rpc('cancel_own_pending_order', { _pedido_id: resumableOrder.pedidoId } as any)
                .setHeader('x-session-id', resumableOrder.sessionId);
        } catch (e) { console.error(e); }
    }
    localStorage.removeItem(ACTIVE_ORDER_KEY);
    setResumableOrder(null);
}, [resumableOrder]);

const handleRetryFromTracking = useCallback(() => {
    if (orderState) setPendingCheckout(orderState);
}, [orderState]);

const handleOrderComplete = useCallback(() => {
    localStorage.removeItem(ACTIVE_ORDER_KEY);
    setOrderState(null);
}, []);

if (recovering) {
    return <MontyLoader />;
}

```

```

if (pendingCheckout) {
  return (
    <EmbeddedCheckout
      pedidoId={pendingCheckout.pedidoId}
      sessionId={pendingCheckout.sessionId ?? ''}
      numeroPedido={pendingCheckout.numeroPedido}
      hasCocina={pendingCheckout.hasCocina}
      hasBebidas={pendingCheckout.hasBebidas}
      total={pendingCheckout.total ?? 0}
      onSuccess={handlePaymentSuccess}
      onCancel={handlePaymentBackToMenu}
    />
  );
}

if (orderState) {
  return (
    <OrderTracking
      pedidoId={orderState.pedidoId}
      numeroPedido={orderState.numeroPedido}
      hasCocina={orderState.hasCocina}
      hasBebidas={orderState.hasBebidas}
      sessionId={orderState.sessionId}
      onNewOrder={handleOrderComplete}
      onRetryPayment={handleRetryFromTracking}
    />
  );
}

// Gate: no local activo => pantalla de selección obligatoria
if (!activeLocal) {
  return <LocalSelector />;
}

return (
  <div className="min-h-screen bg-background">
    <ClientHeader
      localName={activeLocal.nombre}
      mesa={mesaNum ? parseInt(mesaNum) : undefined}
      onCartClick={() => setCartOpen(true)}
      onChangeLocal={() => setShowLocalSelector(true)}
      onFilterClick={() => setFilterOpen(true)}
      onHome={() => { setActiveCategory(null); setActiveSection(null); setCartOpen(false); setFilterOpen(false); }}
    />

    {resumableOrder && (
      <div className="px-4 pt-3 pb-1 max-w-lg mx-auto">
        <div className="flex items-center justify-between gap-3 rounded-xl border border-amber-400/50 bg-amber-50 dark:bg-amber-950/30 px-4 py-3">
          <div className="min-w-0">
            <p className="text-sm font-semibold text-amber-900 dark:text-amber-200">
              Pedido #{resumableOrder.numeroPedido} pendiente de pago
            </p>
            <p className="text-xs text-amber-700 dark:text-amber-400 mt-0.5">
              Tu pedido sigue reservado. ¿Quieres completar el pago?
            </p>
          </div>
          <div className="flex shrink-0 gap-2">
            <button
              onClick={() => setPendingCheckout(resumableOrder)}
              className="rounded-lg bg-primary px-3 py-1.5 text-xs font-semibold text-primary-foreground"
            >
              Pagar
            </button>
            <button
              onClick={handleCancelResumable}
              className="rounded-lg border border-amber-400 px-3 py-1.5 text-xs font-semibold text-amber-800 dark:text-amber-300"
            >
              Cancelar
            </button>
          </div>
        </div>
      </div>
    )}

    {excluded.length > 0 && (
      <div className="px-4 pt-3 max-w-lg mx-auto">
        <div className="flex items-center gap-1.5 flex-wrap">
          <span className="text-[10px] uppercase tracking-wider text-muted-foreground mr-1">
            Excluyendo:
          </span>
          {excluded.map((code) => {
            const a = allAllergens.find((x) => x.codigo === code);
            if (!a) return null;
            return (
              <button
                key={code}
                onClick={() => toggleExcluded(code)}
                className="flex items-center gap-1 pl-1 pr-2 py-0.5 rounded-full bg-destructive/15 border border-destructive/40 text-destructive text-[11px] font-semibold"
              >
                <span aria-hidden={a.icono}</span>

```



```

        {a.nombre}
        <X className="w-3 h-3" />
      </button>
    );
  }}}
</div>
</div>
})

{!activeCategory ? (
  <>
    <div className="px-4 pt-6 pb-6 max-w-lg mx-auto text-center">
      <h2 className="font-display text-3xl font-black leading-tight text-foreground">
        ¿Qué te <span className="text-primary italic">apetece</span> hoy?
      </h2>
      <p className="text-muted-foreground mt-2 text-sm">
        Elige una sección para empezar
      </p>
    </div>
    <div className="pb-8">
      <CategoryGrid categories={categories} onSelect={setActiveCategory} />
    </div>
  </>
) : (
  <>
    <div className="px-4 pt-6 pb-2 max-w-lg mx-auto flex items-center gap-3">
      <button
        onClick={() => { setActiveCategory(null); setActiveSection(null); }}
        className="shrink-0 w-10 h-10 rounded-full bg-card border border-border flex items-center justify-center text-lg
        hover:bg-surface-elevated transition-colors"
        aria-label="Volver"
      >
        ←
      </button>
      <h2 className="font-display text-2xl font-black leading-tight text-foreground">
        {selectedCategory?.nombre ?? 'Menú'}
      </h2>
    </div>

    {(() => {
      const isMontaditos = selectedCategory?.nombre === 'Montaditos';
      const isBebidas = selectedCategory?.nombre === 'Bebidas';
      const isAperitivos = selectedCategory?.nombre === 'Aperitivos';
      const sectionList: readonly string[] | null = isMontaditos
        ? MONTADITO_SECTIONS
        : isBebidas
        ? BEBIDAS_SECTIONS
        : isAperitivos
        ? APERITIVOS_SECTIONS
        : null;
      const isDrink = isBebidas;

      const isMontyAhorro = selectedCategory?.nombre === 'MontyRuedas';

      if (!sectionList) {
        return (
          <div className="px-4 pb-32 max-w-lg mx-auto">

            <div className="flex flex-col gap-2">
              {products.map((product, i) =>
                isMontyAhorro ? (
                  <MontyRuedaCard
                    key={product.id}
                    product={product as any}
                    montaditos={ruedaMontaditos}
                    isProductoAgotado={isProductoAgotado}
                    index={i}
                  />
                ) : (
                  <ProductCard key={product.id} product={product as any} index={i} hideAllergens={isMontyAhorro} agotado=
                    {isProductoAgotado(product.id)} isIngredienteAgotado={isIngredienteAgotado} />
                ),
              )}
            </div>
          </div>
        );
      }

      const available = sectionList.filter((s) =>
        isSeccionAllowed(s) &&
        (products as any[]).some((p) => p.seccion === s)
      );
      const grouped = sectionList
        .filter((sec) => isSeccionAllowed(sec) && (!activeSection || sec === activeSection))
        .map((sec) => ({
          sec,
          items: (products as any[])
            .filter((p) => p.seccion === sec)
            .sort((a, b) => {
              const na = parseInt((a.numero ?? '0').replace(/\D/g, '')) || 0;
              const nb = parseInt((b.numero ?? '0').replace(/\D/g, '')) || 0;

```

```

        return na - nb;
    }},
    )))
    .filter((g) => g.items.length > 0);
const sinSeccion = (products as any[]).filter((p) => !p.seccion);
let idx = 0;
return (
    <>
        <div className="px-4 pb-3 max-w-lg mx-auto">
            <SectionTabs
                sections={available}
                active={activeSection}
                onSelect={setActiveSection}
            />
        </div>
        <div className="px-4 pb-32 max-w-lg mx-auto">
            <div className="flex flex-col gap-6">
                {grouped.map((g) => (
                    <section key={g.sec}>
                        {!activeSection && (
border-primary">
                            <h3 className="font-display text-base font-black uppercase tracking-wider text-primary mb-2 pl-1 border-1-4
                                <span className="pl-2">{g.sec}</span>
                                </h3>
                            ))}
                            <div className="flex flex-col gap-2">
                                {g.items.map((p) => (
                                    <ProductCard
                                        key={p.id}
                                        product={p}
                                        index={idx++}
                                        variant={isDrink ? 'drink' : 'default'}
                                        agotado={isProductoAgotado(p.id)}
                                        isIngredienteAgotado={isIngredienteAgotado}
                                    />
                                ))}
                            </div>
                        </section>
                    ))}
                {(!activeSection) && sinSeccion.length > 0 && (
                    <section>
border-1-4 border-border">
                        <h3 className="font-display text-base font-black uppercase tracking-wider text-muted-foreground mb-2 pl-1
                            <span className="pl-2">Otros</span>
                        </h3>
                        <div className="flex flex-col gap-2">
                            {sinSeccion.map((p) => (
                                <ProductCard
                                    key={p.id}
                                    product={p}
                                    index={idx++}
                                    variant={isDrink ? 'drink' : 'default'}
                                    agotado={isProductoAgotado(p.id)}
                                    isIngredienteAgotado={isIngredienteAgotado}
                                />
                            ))}
                        </div>
                    </section>
                ))}
            </div>
        </div>
    </>
    );
    })()}
    </>
    )}

    <CartSheet
        open={cartOpen}
        onClose={() => setCartOpen(false)}
        onCheckout={handleCheckout}
        isCheckingOut={isCheckingOut}
    />

    <AllergenLegalNoticeFloat />

    <AllergenFilterSheet open={filterOpen} onClose={() => setFilterOpen(false)} />

    <MenuChatBot />
    <AlcoholWarningSheet />

    <WelcomeScreen open={welcomeOpen} onClose={closeWelcome} />
    {showLoader && <MontyLoader message="Preparando el menú" />}

    {showLocalSelector && (
        <div className="fixed inset-0 z-40 bg-background overflow-y-auto">
            <LocalSelector onSelect={() => setShowLocalSelector(false)} />
        </div>
    )}
    </div>
);

```

```

};

export default ClientApp;

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/Index.tsx */
/* ===== */
// Update this page (the content is just a fallback if you fail to update the page)

// IMPORTANT: Fully REPLACE this with your own code
const PlaceholderIndex = () => {
  // PLACEHOLDER: Replace this entire return statement with the user's app.
  // The inline background color is intentionally not part of the design system.
  return (
    <div className="flex min-h-screen items-center justify-center" style={{ backgroundColor: '#fcfbf8' }}>
      
    </div>
  );
};

const Index = PlaceholderIndex;

export default Index;

/* ===== */
/* FICHERO: montadito-magic-flow/src/pages/NotFound.tsx */
/* ===== */
import { useLocation } from "react-router-dom";
import { useEffect } from "react";

const NotFound = () => {
  const location = useLocation();

  useEffect(() => {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-muted">
      <div className="text-center">
        <h1 className="mb-4 text-4xl font-bold">404</h1>
        <p className="mb-4 text-xl text-muted-foreground">Oops! Page not found</p>
        <a href="/" className="text-primary underline hover:text-primary/90">
          Return to Home
        </a>
      </div>
    </div>
  );
};

export default NotFound;

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AlcoholWarningSheet.tsx */
/* ===== */
import { AnimatePresence, motion } from 'framer-motion';
import { AlertTriangle } from 'lucide-react';
import { useAgeGate } from '@lib/cart-store';

export function AlcoholWarningSheet() {
  const pending = useAgeGate((s) => s.pending);
  const confirm = useAgeGate((s) => s.confirm);
  const cancel = useAgeGate((s) => s.cancel);
  const open = !!pending;

  return (
    <AnimatePresence>
      {open && (
        <>
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="fixed inset-0 bg-black/70 z-[60]"
            onClick={cancel}
          />
          <motion.div
            initial={{ y: '100%' }}
            animate={{ y: 0 }}
            exit={{ y: '100%' }}
            transition={{ type: 'spring', damping: 30, stiffness: 300 }}
            className="fixed bottom-0 left-0 right-0 z-[61] rounded-t-3xl bg-popover border-t border-border-subtle p-6 pb-8"
            role="alertdialog"
            aria-labelledby="alcohol-title"
          >
            <div className="flex justify-center pt-1 pb-4">
              <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
            </div>
            <div className="flex flex-col items-center text-center max-w-sm mx-auto">

```

```

        <div className="w-16 h-16 rounded-full bg-destructive/15 flex items-center justify-center mb-4">
          <AlertTriangle className="w-8 h-8 text-destructive" />
        </div>
        <h2 id="alcohol-title" className="font-display text-xl font-bold mb-2">
          Verificación de edad
        </h2>
        <p className="text-sm text-muted-foreground leading-relaxed mb-6">
          La venta de alcohol a menores de 18 años está prohibida por ley en España.
        </p>
        <div className="flex flex-col gap-2 w-full">
          <button
            onClick={confirm}
            className="w-full py-3.5 rounded-2xl bg-destructive text-destructive-foreground font-bold text-sm"
          >
            Confirmo que soy mayor de 18 años
          </button>
          <button
            onClick={cancel}
            className="w-full py-3 rounded-2xl bg-surface text-foreground font-semibold text-sm"
          >
            Cancelar
          </button>
        </div>
      </motion.div>
    </>
  )}
</AnimatePresence>
);
}

/* ===== */
/* FICHERO: montadito-magic-flow/src/components/client/AllergenFilterSheet.tsx */
/* ===== */
import { motion, AnimatePresence } from 'framer-motion';
import { X, Check } from 'lucide-react';
import { useAllergens } from '@hooks/use-allergens';
import { useAllergenFilter } from '@lib/allergen-filter';
import { AllergenIcon } from './AllergenIcon';
import { AllergenLegalNotice } from './AllergenLegalNotice';

interface Props {
  open: boolean;
  onClose: () => void;
}

export function AllergenFilterSheet({ open, onClose }: Props) {
  const { data: alergenos = [] } = useAllergens();
  const excluded = useAllergenFilter((s) => s.excluded);
  const toggle = useAllergenFilter((s) => s.toggle);
  const clear = useAllergenFilter((s) => s.clear);
  const hideMode = useAllergenFilter((s) => s.hideMode);
  const setHideMode = useAllergenFilter((s) => s.setHideMode);

  return (
    <AnimatePresence>
      {open && (
        <>
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="fixed inset-0 bg-black/60 z-40"
            onClick={onClose}
          />
          <motion.div
            initial={{ y: '100%' }}
            animate={{ y: 0 }}
            exit={{ y: '100%' }}
            transition={{ type: 'spring', damping: 30, stiffness: 300 }}
            className="fixed bottom-0 left-0 right-0 z-50 max-h-[88vh] rounded-t-3xl bg-popover border-t border-border-subtle flex
flex-col"
          >
            <div className="flex justify-center pt-3 pb-2">
              <div className="w-10 h-1 rounded-full bg-muted-foreground/30" />
            </div>

            <div className="flex items-center justify-between px-6 pb-2">
              <div>
                <h2 className="font-display text-xl font-bold">Filtrar alérgenos</h2>
                <p className="text-xs text-muted-foreground mt-0.5">
                  Selecciona los que quieres <span className="font-semibold text-destructive">excluir</span>.
                </p>
              </div>
              <button onClick={onClose} className="p-2 rounded-full hover:bg-surface transition-colors">

```

[... páginas intermedias del código omitidas en este extracto ...]

```

        ('B28','gluten'),      -- Heineken 0.0
        ('B29','gluten'),      -- Desperados
        ('B30','gluten'),      -- Paulaner
        ('B31','sulfitos'),     -- Buenos Días Ribera de Duero
        ('B32','sulfitos'),     -- Cune Rosado Navarra
        ('B33','sulfitos'),     -- Cune Blanco Verdejo
        ('B34','sulfitos'),     -- Cune Tinto Crianza Rioja
        ('B36','lacteos'),      -- Café con Ruavieja
        ('B39','lacteos')      -- Batido
    )
ON CONFLICT DO NOTHING;

-- =====
-- 4. APERITIVOS
-- Correcciones respecto a migración anterior:
-- - Patatas fritas 4 salsas: se añade gluten(X) confirmado en PDF
-- - Palomitas de pollo: se elimina huevos (era trazas *)
-- - Palomitas de queso gouda: se elimina huevos (era trazas *)
-- - Nachos con bacon: se añade sulfitos(X) confirmado en PDF
-- - Aceitunas manzanilla: cubierta por patrón ILIKE '%aceituna%'
-- =====

-- Gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'gluten') a
WHERE p.id IN (
    '9ab4f327-1535-4103-b292-2068d18d3d50', -- Patatas fritas 4 salsas
    'fb40b377-e42f-4d3d-b347-4dcb1cf8ddcd', -- Palomitas de pollo
    '17dc39bf-d4ba-4728-91eb-f7c4cba31442', -- Palomitas de queso gouda
    'a83faca2-9849-4c16-a1ea-f682fd51b923', -- Salchichas 4 salsas
    '0aa86602-8494-437b-9e60-5f95d71d8900' -- Cheesy Cheetos Bites
)
ON CONFLICT DO NOTHING;

-- Huevos (solo productos con huevos confirmados X, no trazas)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'huevos') a
WHERE p.id IN (
    '9ab4f327-1535-4103-b292-2068d18d3d50', -- Patatas fritas 4 salsas
    'a83faca2-9849-4c16-a1ea-f682fd51b923' -- Salchichas 4 salsas
)
ON CONFLICT DO NOTHING;

-- Lácteos – productos con UUID conocido
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.id IN (
    '3ca1c5df-1578-419d-920d-25ba5012e8b6', -- Patatas fritas cheddar y bacon
    '17dc39bf-d4ba-4728-91eb-f7c4cba31442', -- Palomitas de queso gouda
    '0aa86602-8494-437b-9e60-5f95d71d8900' -- Cheesy Cheetos Bites
)
ON CONFLICT DO NOTHING;

-- Lácteos – nachos (ambas variantes: cheddar+bacon y cheddar+guacamole)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
    AND p.nombre ILIKE '%nachos%'
    AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Sulfitos – patatas fritas cheddar y bacon
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.id = '3ca1c5df-1578-419d-920d-25ba5012e8b6'
ON CONFLICT DO NOTHING;

-- Sulfitos – nachos con bacon (confirmado en PDF)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
    AND p.nombre ILIKE '%nachos%'
    AND p.nombre ILIKE '%bacon%'
    AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Sulfitos – aceitunas (de la abuela + manzanilla)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id

```

```

FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'sulfitos') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
      AND p.nombre ILIKE '%aceituna%'
      AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Soja
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'soja') a
WHERE p.id = 'a83faca2-9849-4c16-a1ea-f682fd51b923' -- Salchichas 4 salsas
ON CONFLICT DO NOTHING;

-- Pescado – gildas (boquerón y anchoa)
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'pescado') a
WHERE p.categoria_id = '32447a0c-6911-4c3f-98d8-88a58d3e15ca'
      AND p.nombre ILIKE '%gilda%'
      AND p.disponible = true
ON CONFLICT DO NOTHING;

-- Cucurucho de patatas chips → sin alérgenos confirmados

-- =====
-- 5. RACIONES
-- Correcciones respecto a migración anterior:
-- - Croquetas de Jamón: se elimina huevos (trazas * en PDF)
-- - Alitas de pollo BBQ: se elimina huevos, se añade apio
-- - Patatas fritas Bravioli: se elimina huevos (trazas * en PDF)
-- - Tiras de pollo: se elimina huevos (no confirmado en PDF)
-- - Torreznos: sin alérgenos confirmados (igual que antes)
-- =====

-- Gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'gluten') a
WHERE p.id IN (
  '3795cecd-4939-49fa-99a5-46b3acf1cb42', -- Croquetas de Jamón
  'f105cb80-8834-4b88-b8bd-a13e0fcb607', -- Croquetas de Mac&Cheese
  '2c747f14-7fad-440b-8947-7c913fbfc644', -- Alitas de pollo
  'a40bd701-89cc-4457-9a6b-f179eeb6ad20' -- Tiras de pollo
)
ON CONFLICT DO NOTHING;

-- Lácteos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'lacteos') a
WHERE p.id IN (
  '3795cecd-4939-49fa-99a5-46b3acf1cb42', -- Croquetas de Jamón
  'f105cb80-8834-4b88-b8bd-a13e0fcb607' -- Croquetas de Mac&Cheese
)
ON CONFLICT DO NOTHING;

-- Apio – alitas de pollo sabor BBQ
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN (SELECT id FROM alergenos WHERE codigo = 'apio') a
WHERE p.id = '2c747f14-7fad-440b-8947-7c913fbfc644'
ON CONFLICT DO NOTHING;

-- Patatas fritas Bravioli (52f15185): huevos son trazas (*) → sin alérgenos confirmados
-- Torreznos: sin alérgenos confirmados en PDF

-- =====
-- 6. ENSALADAS
-- Correcciones respecto a migración anterior:
-- - TexMex: se eliminan soja y apio (trazas * en PDF); solo gluten+mostaza
-- - Campera: se elimina sulfitos (trazas * en PDF); solo gluten+huevos+lacteos
-- =====

-- TexMex
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.id = 'bd45575b-709f-4f36-9e97-00c313cdb71f'
      AND a.codigo IN ('gluten', 'mostaza')
ON CONFLICT DO NOTHING;

-- Campera
INSERT INTO producto_alergenos (producto_id, alergeno_id)

```

```

SELECT p.id, a.id
FROM menu_productos p
CROSS JOIN alergenos a
WHERE p.id = '3d61ba4a-c009-4d0c-bd74-aa20ea5720e4'
  AND a.codigo IN ('gluten', 'huevos', 'lacteos')
ON CONFLICT DO NOTHING;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260613035000_security_constraints.sql */
/* ===== */
-- Constraints de seguridad: impedir importes negativos o cero en pedidos
ALTER TABLE public.pedidos
  ADD CONSTRAINT pedidos_total_non_negative CHECK (total >= 0);

ALTER TABLE public.pedido_items
  ADD CONSTRAINT items_cantidad_positive CHECK (cantidad > 0),
  ADD CONSTRAINT items_precio_unitario_positive CHECK (precio_unitario > 0);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614000000_precios_jarras.sql */
/* ===== */
-- Corrección de precios de jarras – Junio 2026
--
-- Lógica de precios (precio = Quijote; Sancho = precio + 0.50€):
-- Cerveza Premium → Quijote 2.00 € / Sancho 2.50 € → precio = 2.00
-- Jarras Heladas → Quijote 1.50 € / Sancho 2.00 € → precio = 1.50

-- Cerveza Premium: precio base = 2.00 €
UPDATE public.menu_productos
SET precio = 2.00
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND seccion = 'Cerveza Premium';

-- Jarras Heladas: precio base = 1.50 € (Quijote)
UPDATE public.menu_productos
SET precio = 1.50
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND seccion = 'Jarras Heladas';

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614010000_precios_agua_infusion.sql */
/* ===== */
-- Corrección de precios puntuales – Junio 2026
-- Agua mineral: 2.20 € (antes 2.00 €)
-- Infusiones: 1.30 € (confirmación, sin cambio)

-- Agua
UPDATE public.menu_productos
SET precio = 2.20
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND nombre ILIKE '%agua%';

-- Infusiones (manzanilla, tila, poleo...)
UPDATE public.menu_productos
SET precio = 1.30
WHERE categoria_id = '2a9b31b6-8752-4bbd-8950-3219458f7fa9'
  AND (
    nombre ILIKE '%infusi%'
    OR nombre ILIKE '%manzanilla%'
    OR nombre ILIKE '%tila%'
    OR nombre ILIKE '%poleo%'
  );

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614020000_alergenos_montaditos_1_9.sql */
/* ===== */
-- FIX: Alérgenos montaditos 01-09
-- La migración 20260613020000 usaba numero = '01'..'09' (con cero),
-- pero la BD almacena numero = '1'..'9' (sin cero), por lo que el INSERT
-- no hizo match en esos productos. Se repite usando numero::integer = N.
-- Fuente: PDF 100M_Alergenos_MARZO_2026.pdf
-- =====

-- 1. Limpiar posibles entradas huérfanas de 1-9 (idempotente)
DELETE FROM producto_alergenos
WHERE producto_id IN (
  SELECT id FROM menu_productos
  WHERE categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND disponible = true
    AND numero ~ '^\\d+$'
    AND numero::integer BETWEEN 1 AND 9
);

-- 01 Jamón Gran Reserva y aceite de oliva – gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'

```



```

    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 1
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

-- 02 Tortilla de patatas y tomate - gluten, huevos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 2
    AND a.codigo IN ('gluten','huevos')
ON CONFLICT DO NOTHING;

-- 03 Pulled pork BBQ - gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 3
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

-- 04 Pollo y salsa alioli - gluten, huevos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 4
    AND a.codigo IN ('gluten','huevos')
ON CONFLICT DO NOTHING;

-- 05 Carrillera al vino tinto - gluten, lacteos, sulfitos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 5
    AND a.codigo IN ('gluten','lacteos','sulfitos')
ON CONFLICT DO NOTHING;

-- 06 Calamarcitos y mayonesa - gluten, huevos, moluscos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 6
    AND a.codigo IN ('gluten','huevos','moluscos')
ON CONFLICT DO NOTHING;

-- 07 Pollo kebab y salsa BBQ - gluten, apio
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 7
    AND a.codigo IN ('gluten','apio')
ON CONFLICT DO NOTHING;

-- 08 Bacon ahumado y queso madurado - gluten, lacteos
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 8
    AND a.codigo IN ('gluten','lacteos')
ON CONFLICT DO NOTHING;

-- 09 Torreznos y salsa brava - gluten
INSERT INTO producto_alergenos (producto_id, alergeno_id)
SELECT p.id, a.id FROM menu_productos p CROSS JOIN alergenos a
WHERE p.categoria_id = 'f3b118eb-c117-4a4c-9b61-7d10d9a3ebf9'
    AND p.disponible = true AND p.numero ~ '^\\d+$' AND p.numero::integer = 9
    AND a.codigo IN ('gluten')
ON CONFLICT DO NOTHING;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614030000_local_restricciones.sql */
/* ===== */
-- =====
-- FEATURE: Restricciones por local
-- Permite a cada local desactivar productos o secciones, con soporte
-- opcional para restricciones horarias (hora_limite: no disponible desde esa hora).
-- =====

CREATE TABLE IF NOT EXISTS public.local_restricciones (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    local_id    UUID          NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    producto_id UUID          REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    seccion     TEXT,
    -- null → siempre no disponible en este local
    -- valor → no disponible A PARTIR de esa hora (ej. '17:00')
    hora_limite TIME,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_restriccion_target CHECK (producto_id IS NOT NULL OR seccion IS NOT NULL)
);

CREATE UNIQUE INDEX IF NOT EXISTS local_restricciones_producto_uq
ON public.local_restricciones (local_id, producto_id)

```

```

WHERE producto_id IS NOT NULL;

CREATE UNIQUE INDEX IF NOT EXISTS local_restricciones_seccion_uq
ON public.local_restricciones (local_id, seccion)
WHERE seccion IS NOT NULL;

ALTER TABLE public.local_restricciones ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone can view restricciones"
ON public.local_restricciones FOR SELECT USING (true);

CREATE POLICY "Admins can manage restricciones"
ON public.local_restricciones FOR ALL
USING (public.has_role(auth.uid(), 'admin'));

-- — Restricciones del local de Móstoles —————

DO $$
DECLARE
    v_local UUID;
BEGIN
    SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
    IF v_local IS NULL THEN
        RAISE NOTICE 'Local mostoles-constitucion no encontrado, saltando restricciones.';
        RETURN;
    END IF;

    -- 1. Sección "Tardeo Chill" completa
    INSERT INTO public.local_restricciones (local_id, seccion)
    VALUES (v_local, 'Tardeo Chill')
    ON CONFLICT DO NOTHING;

    -- 2. "Café e Infusiones" no disponible a partir de las 17:00
    INSERT INTO public.local_restricciones (local_id, seccion, hora_limite)
    VALUES (v_local, 'Café e Infusiones', '17:00')
    ON CONFLICT DO NOTHING;

    -- 3. Appletiser
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true AND nombre ILIKE '%appletiser%'
    ON CONFLICT DO NOTHING;

    -- 4. Batidos que no sean de chocolate
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%batido%'
        AND nombre NOT ILIKE '%chocolate%'
    ON CONFLICT DO NOTHING;

    -- 5. Monster que no sea Ultra White / Blanco
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%monster%'
        AND nombre !~* 'ultra|blanco|white'
    ON CONFLICT DO NOTHING;

    -- 6. Heineken con jarra (no Heineken 0.0 / sin alcohol)
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%heineken%'
        AND nombre !~* '0[.]0|sin\s*alcohol|00'
    ON CONFLICT DO NOTHING;

    -- 7. Tercios Cruzcampo (no Cruzcampo Sin Gluten)
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%cruzcampo%'
        AND nombre NOT ILIKE '%sin gluten%'
    ON CONFLICT DO NOTHING;

    -- 8. Spritz y Petroni
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND (nombre ILIKE '%spritz%' OR nombre ILIKE '%petroni%')
    ON CONFLICT DO NOTHING;

    -- 9. Desperados
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
        AND nombre ILIKE '%desperados%'
    ON CONFLICT DO NOTHING;

END;
$$;

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614040000_mostoles_seed_y_restricciones.sql */
/* ===== */
-- =====
-- Insertar local Mostoles (si no existe) y sus restricciones de carta.
-- La migración 20260614030000 detectó que el slug 'mostoles-constitucion'
-- no estaba en la BD; se re-inserta aquí de forma idempotente.
-- =====

-- — 1. Franchisee (idempotente) —————

INSERT INTO public.franchisees (nombre, email, application_fee_percent)
VALUES ('Mostoles', 'mostoles@hat3x.com', 0)
ON CONFLICT (email) DO NOTHING;

-- — 2. Local (idempotente) —————
-- franchisee_id es NOT NULL (añadido en migración 20260522).

INSERT INTO public.locales (nombre, ciudad, direccion, slug, lat, lng, activo, franchisee_id)
SELECT
  'Mostoles - Av. de la Constitución',
  'Mostoles',
  'Av. de la Constitución, 23, 28931 Mostoles, Madrid',
  'mostoles-constitucion',
  40.3223,
  -3.8652,
  true,
  f.id
FROM public.franchisees f
WHERE f.email = 'mostoles@hat3x.com'
ON CONFLICT (slug) DO NOTHING;

-- — 2. Restricciones —————

DO $$
DECLARE
  v_local UUID;
BEGIN
  SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
  IF v_local IS NULL THEN
    RAISE EXCEPTION 'Local mostoles-constitucion no encontrado incluso después del INSERT';
  END IF;

  -- Sección "Tardeo Chill" completa
  INSERT INTO public.local_restricciones (local_id, seccion)
  VALUES (v_local, 'Tardeo Chill')
  ON CONFLICT DO NOTHING;

  -- "Café e Infusiones" no disponible a partir de las 17:00
  INSERT INTO public.local_restricciones (local_id, seccion, hora_limite)
  VALUES (v_local, 'Café e Infusiones', '17:00')
  ON CONFLICT DO NOTHING;

  -- Appletiser
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true AND nombre ILIKE '%appletiser%'
  ON CONFLICT DO NOTHING;

  -- Batidos que no sean de chocolate
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%batido%'
    AND nombre NOT ILIKE '%chocolate%'
  ON CONFLICT DO NOTHING;

  -- Monster que no sea Ultra White / Blanco
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%monster%'
    AND nombre !~* 'ultra|blanco|white'
  ON CONFLICT DO NOTHING;

  -- Heineken (no Heineken 0.0 / sin alcohol)
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%heineken%'
    AND nombre !~* '0[.]0|sin\s*alcohol|00'
  ON CONFLICT DO NOTHING;

  -- Tercios Cruzcampo (no Cruzcampo Sin Gluten)
  INSERT INTO public.local_restricciones (local_id, producto_id)
  SELECT v_local, id FROM public.menu_productos
  WHERE disponible = true
    AND nombre ILIKE '%cruzcampo%'
    AND nombre NOT ILIKE '%sin gluten%'
  ON CONFLICT DO NOTHING;

```

```

-- Spritz y Petroni
INSERT INTO public.local_restricciones (local_id, producto_id)
SELECT v_local, id FROM public.menu_productos
WHERE disponible = true
AND (nombre ILIKE '%spritz%' OR nombre ILIKE '%petroni%')
ON CONFLICT DO NOTHING;

-- Desperados
INSERT INTO public.local_restricciones (local_id, producto_id)
SELECT v_local, id FROM public.menu_productos
WHERE disponible = true
AND nombre ILIKE '%desperados%'
ON CONFLICT DO NOTHING;

END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614050000_mostoles_restricciones_jarras.sql */
/* ===== */
-- =====
-- Mostoles – restricciones precisas de secciones de jarras/botella.
-- En cada sección se ocultan todos los productos EXCEPTO los aprobados.
-- =====

DO $$
DECLARE
    v_local UUID;
BEGIN
    SELECT id INTO v_local FROM public.locales WHERE slug = 'mostoles-constitucion';
    IF v_local IS NULL THEN
        RAISE EXCEPTION 'Local mostoles-constitucion no encontrado';
    END IF;

    -- — JARRAS HELADAS —————
    -- Solo: Jarra Cruzcampo Especial · Jarra Ladrón de Verano
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Jarras Heladas'
    AND NOT (
        nombre ILIKE '%cruzcampo%especial%'
        OR nombre ILIKE '%especial%cruzcampo%'
        OR nombre ILIKE '%ladr_n%verano%'
        OR nombre ILIKE '%verano%ladr_n%'
        OR nombre ILIKE '%ladron%verano%'
        OR nombre ILIKE '%verano%ladron%'
    )
    ON CONFLICT DO NOTHING;

    -- — CERVEZA PREMIUM —————
    -- Solo: Jarra Cruzcampo Radler · Jarra Ladrón de Manzanas · Jarra El Águila Dorada
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Cerveza Premium'
    AND NOT (
        nombre ILIKE '%cruzcampo%radler%'
        OR nombre ILIKE '%radler%cruzcampo%'
        OR nombre ILIKE '%ladr_n%manzana%'
        OR nombre ILIKE '%manzana%ladr_n%'
        OR nombre ILIKE '%ladron%manzana%'
        OR nombre ILIKE '%manzana%ladron%'
        OR nombre ILIKE '%_guila%dorada%'
        OR nombre ILIKE '%dorada%guila%'
    )
    ON CONFLICT DO NOTHING;

    -- — CERVEZA EN BOTELLA —————
    -- Solo: Heineken 0.0 · Paulaner
    INSERT INTO public.local_restricciones (local_id, producto_id)
    SELECT v_local, id FROM public.menu_productos
    WHERE disponible = true
    AND seccion = 'Cerveza en Botella'
    AND NOT (
        nombre ILIKE '%heineken%'
        OR nombre ILIKE '%paulaner%'
    )
    ON CONFLICT DO NOTHING;

END;
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614060000_activar_batido_chocolate.sql */
/* ===== */
-- Activa el batido de chocolate si estaba desactivado en la carta global.
UPDATE public.menu_productos
SET disponible = true
WHERE (nombre ILIKE '%batido%chocolate%' OR nombre ILIKE '%chocolate%batido%')

```

```

AND disponible = false;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614070000_mostoles_desbloquear_batido.sql */
/* ===== */
-- El producto se llama "Batido" (sin especificar sabor).
-- La restricción anterior lo bloqueó porque no contiene "chocolate" en el nombre.
-- Se elimina esa entrada para que el Batido vuelva a ser visible en Mostoles.

DELETE FROM public.local_restricciones
WHERE local_id = (SELECT id FROM public.locales WHERE slug = 'mostoles-constitucion')
AND producto_id IN (
    SELECT id FROM public.menu_productos
    WHERE nombre ILIKE '%batido%'
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260614080000_mostoles_desbloquear_jarras_cruzcampo.sql */
/* ===== */
-- La restricción "tercios Cruzcampo" de la migración 040000 bloqueó también
-- las Jarras Cruzcampo (Especial y Radler), que SÍ deben estar disponibles.
-- Se eliminan solo las restricciones de productos Cruzcampo que son jarras.

DELETE FROM public.local_restricciones
WHERE local_id = (SELECT id FROM public.locales WHERE slug = 'mostoles-constitucion')
AND producto_id IN (
    SELECT id FROM public.menu_productos
    WHERE nombre ILIKE '%cruzcampo%'
    AND (
        nombre ILIKE '%jarra%'
        OR nombre ILIKE '%especial%'
        OR nombre ILIKE '%radler%'
    )
);

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260616090000_aperitivos_destino_bebidas.sql */
/* ===== */
--
-- FIX: aperitivos servidos en barra (aceitunas, gilda, cucurucho) deben ir a la
-- pista de "bebidas" (barra/caja), NO a cocina.
--
-- Causa raíz: resolve_pedido_item_destino() solo mandaba a 'bebidas' la categoría
-- 'Bebidas'. Los aperitivos están en la categoría 'Aperitivos', así que caían en
-- el ELSE → 'cocina'. Además el trigger set_pedido_item_destino_before_write pisa
-- el destino que envía el cliente con el resultado de esta función, por lo que
-- daba igual lo que mandara la app: el aperitivo siempre acababa en cocina.
--
-- Solución: añadir el reconocimiento de aperitivos por nombre en la función.
-- El trigger sync_pedido_sections (ya existente) recalcula estado_cocina /
-- estado_bebidas / tipo a partir del destino, así que el resto del flujo se
-- ajusta solo. No hace falta tocar nada más.
--
CREATE OR REPLACE FUNCTION public.resolve_pedido_item_destino(_producto_id uuid)
RETURNS text
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path TO 'public'
AS $$
    SELECT CASE
        WHEN mc.nombre = 'Bebidas' THEN 'bebidas'
        -- Aperitivos servidos en barra: se entregan en caja, igual que las bebidas.
        WHEN lower(mp.nombre) LIKE '%aceituna%'
            OR lower(mp.nombre) LIKE '%gilda%'
            OR lower(mp.nombre) LIKE '%cucurucho%' THEN 'bebidas'
        ELSE 'cocina'
    END
    FROM public.menu_productos mp
    LEFT JOIN public.menu_categorias mc ON mc.id = mp.categoria_id
    WHERE mp.id = _producto_id
$$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260617120000_pedido_items_variante.sql */
/* ===== */
--
-- Variante del producto en el ítem del pedido (ej. "Jarra Quijote" / "Jarra
-- Sancho", "Boquerón" / "Anchoa", "BBQ" / "Brava", etc.)
--
-- Hasta ahora pedido_items solo guardaba producto_id, así que el dashboard
-- mostraba el nombre base y se perdía el tamaño/variante elegido por el cliente.
-- Añadimos una columna de texto libre para conservar esa elección.
--
ALTER TABLE public.pedido_items
ADD COLUMN IF NOT EXISTS variante text;

```

```

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260618120000_ingredientes_agotados.sql */
/* ===== */

-- SISTEMA DE INGREDIENTES Y "AGOTADOS" POR LOCAL
--
-- Objetivo: que el personal pueda marcar agotado:
-- (a) un INGREDIENTE (incluye los panes) -> los montaditos/ensaladas que lo
--     lleven aparecen en la app del cliente con aviso "AGOTADO" (no se ocultan).
-- (b) un PRODUCTO entero (aperitivos, raciones, bebidas, cookies, ruedas) ->
--     ese producto aparece con aviso "AGOTADO".
-- Todo es POR LOCAL (cada restaurante marca lo suyo).
--
-- Catálogo de ingredientes (tipo 'ingrediente' o 'pan')
CREATE TABLE IF NOT EXISTS public.ingredientes (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre      text NOT NULL UNIQUE,
    tipo        text NOT NULL DEFAULT 'ingrediente' CHECK (tipo IN ('ingrediente','pan')),
    created_at  timestampz NOT NULL DEFAULT now()
);

-- Qué ingredientes lleva cada producto (solo montaditos + ensaladas)
CREATE TABLE IF NOT EXISTS public.producto_ingredientes (
    producto_id  uuid NOT NULL REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    ingrediente_id uuid NOT NULL REFERENCES public.ingredientes(id) ON DELETE CASCADE,
    PRIMARY KEY (producto_id, ingrediente_id)
);

-- Ingredientes agotados por local (la fila existe = está agotado en ese local)
CREATE TABLE IF NOT EXISTS public.local_ingredientes_agotados (
    local_id     uuid NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    ingrediente_id uuid NOT NULL REFERENCES public.ingredientes(id) ON DELETE CASCADE,
    agotado_at   timestampz NOT NULL DEFAULT now(),
    PRIMARY KEY (local_id, ingrediente_id)
);

-- Productos enteros agotados por local (aperitivos/raciones/bebidas/cookies/ruedas)
CREATE TABLE IF NOT EXISTS public.local_productos_agotados (
    local_id     uuid NOT NULL REFERENCES public.locales(id) ON DELETE CASCADE,
    producto_id  uuid NOT NULL REFERENCES public.menu_productos(id) ON DELETE CASCADE,
    agotado_at   timestampz NOT NULL DEFAULT now(),
    PRIMARY KEY (local_id, producto_id)
);

CREATE INDEX IF NOT EXISTS idx_prod_ing_ingredient ON public.producto_ingredientes(ingrediente_id);
CREATE INDEX IF NOT EXISTS idx_local_ing_agot_local ON public.local_ingredientes_agotados(local_id);
CREATE INDEX IF NOT EXISTS idx_local_prod_agot_local ON public.local_productos_agotados(local_id);

-- --- RLS
ALTER TABLE public.ingredientes          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.producto_ingredientes ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.local_ingredientes_agotados ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.local_productos_agotados  ENABLE ROW LEVEL SECURITY;

-- Lectura para todos (cliente anónimo + dashboard)
CREATE POLICY "ingredientes_read"          ON public.ingredientes          FOR SELECT USING (true);
CREATE POLICY "producto_ingredientes_read" ON public.producto_ingredientes FOR SELECT USING (true);
CREATE POLICY "local_ing_agotados_read"    ON public.local_ingredientes_agotados FOR SELECT USING (true);
CREATE POLICY "local_prod_agotados_read"   ON public.local_productos_agotados FOR SELECT USING (true);

-- Escritura (marcar/desmarcar agotado) solo para staff autenticado (dashboard)
CREATE POLICY "local_ing_agotados_write" ON public.local_ingredientes_agotados
    FOR ALL TO authenticated USING (true) WITH CHECK (true);
CREATE POLICY "local_prod_agotados_write" ON public.local_productos_agotados
    FOR ALL TO authenticated USING (true) WITH CHECK (true);

-- --- Realtime (para que la app del cliente se actualice al instante) ---
ALTER TABLE public.local_ingredientes_agotados REPLICA IDENTITY FULL;
ALTER TABLE public.local_productos_agotados    REPLICA IDENTITY FULL;
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname='supabase_realtime' AND tablename='local_ingredientes_agotados')
    THEN
        EXECUTE 'ALTER PUBLICATION supabase_realtime ADD TABLE public.local_ingredientes_agotados';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname='supabase_realtime' AND tablename='local_productos_agotados') THEN
        EXECUTE 'ALTER PUBLICATION supabase_realtime ADD TABLE public.local_productos_agotados';
    END IF;
END $$;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260620120000_fix_precio_unitario_seguridad.sql */
/* ===== */

-- FIX DE SEGURIDAD (HIGH): manipulación de precios en el pago.
--
-- Problema: el cliente inserta pedido_items con `precio_unitario` arbitrario y
-- create-payment-intent recalcula el total a partir de ese valor. Un atacante

```

```

-- podía insertar un producto real con precio_unitario = 0.01 y pagar 1 céntimo.
--
-- Solución: forzar en la BD el precio real desde menu_productos.precio.
--   - Items SIN variante -> precio_unitario = precio del producto (autoritativo).
--   - Items CON variante -> no se permite por debajo del precio base
--                               (se conserva el recargo de la variante, p. ej. Sancho).
-- Se aplica en el servidor, así que ya no depende de lo que envíe el cliente.
--
CREATE OR REPLACE FUNCTION public.enforce_precio_unitario()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    v_base numeric;
BEGIN
    SELECT precio INTO v_base FROM public.menu_productos WHERE id = NEW.producto_id;
    IF v_base IS NULL THEN
        RETURN NEW; -- producto inexistente: no debería ocurrir
    END IF;

    IF NEW.variante IS NULL THEN
        NEW.precio_unitario := v_base; -- precio autoritativo
    ELSE
        NEW.precio_unitario := GREATEST(COALESCE(NEW.precio_unitario, v_base), v_base); -- nunca por debajo del base
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_enforce_precio_unitario ON public.pedido_items;
CREATE TRIGGER trg_enforce_precio_unitario
BEFORE INSERT OR UPDATE OF precio_unitario, producto_id ON public.pedido_items
FOR EACH ROW
EXECUTE FUNCTION public.enforce_precio_unitario();

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260620130000_security_hardening.sql */
/* ===== */
-- SECURITY HARDENING - 2026-06-20
-- SEC-003: secreto de push fuera del código (tabla protegida app_secrets)
-- SEC-004: rate limit por sesión/IP en chat-menu
-- SEC-006: escrituras de "agotados" acotadas al local del staff
-- SEC-008: la consulta a franchisees devuelve [] en vez de error que filtra
--           el nombre de una función interna
-- =====
-- — SEC-003: secreto de push en tabla protegida (nunca en el repo) —
CREATE TABLE IF NOT EXISTS public.app_secrets (
    key    text PRIMARY KEY,
    value  text NOT NULL
);
ALTER TABLE public.app_secrets ENABLE ROW LEVEL SECURITY;
-- Sin políticas => anon/authenticated NO pueden leer ni escribir vía API.
REVOKE ALL ON public.app_secrets FROM anon, authenticated;

-- Placeholder. El valor REAL se pone con un UPDATE aparte (no versionado):
-- UPDATE public.app_secrets SET value = '<secreto-real>' WHERE key = 'push_notify_secret';
INSERT INTO public.app_secrets(key, value)
VALUES ('push_notify_secret', 'REEMPLAZAR_POR_SECRETO_REAL')
ON CONFLICT (key) DO NOTHING;

-- El trigger lee el secreto de la tabla (SECURITY DEFINER ignora RLS).
CREATE OR REPLACE FUNCTION public.notify_order_listo()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_secret TEXT;
    v_fired  BOOLEAN := FALSE;
BEGIN
    SELECT value INTO v_secret FROM public.app_secrets WHERE key = 'push_notify_secret';

    IF NEW.estado_cocina = 'listo' AND OLD.estado_cocina IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado_bebidas = 'listo' AND OLD.estado_bebidas IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;
    IF NEW.estado        = 'listo' AND OLD.estado        IS DISTINCT FROM 'listo' THEN v_fired := TRUE; END IF;

    IF v_fired THEN
        BEGIN
            PERFORM net.http_post(
                url      := 'https://znmhqnmmtkkaillwfscu.supabase.co/functions/v1/notify-order-ready',
                headers := jsonb_build_object('Content-Type','application/json','x-push-notify-secret', v_secret),
                body     := jsonb_build_object('pedido_id',NEW.id,'session_id',NEW.session_id,'numero_pedido',NEW.numero_pedido)::text
            );
        EXCEPTION WHEN OTHERS THEN
            NULL; -- push best-effort: no debe abortar el UPDATE del pedido
        END;
    END IF;
END IF;

```

```

RETURN NEW;
END; $$;

-- -- SEC-004: rate limit (chat-menu) -----
CREATE TABLE IF NOT EXISTS public.chat_rate_limit (
    key          text PRIMARY KEY,
    count        int  NOT NULL DEFAULT 0,
    window_start timestamptz NOT NULL DEFAULT now()
);
ALTER TABLE public.chat_rate_limit ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.chat_rate_limit FROM anon, authenticated;

-- Devuelve TRUE si se permite la petición (y la contabiliza), FALSE si se supera.
CREATE OR REPLACE FUNCTION public.check_rate_limit(p_key text, p_limit int, p_window_secs int)
RETURNS boolean LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_count int;
    v_start timestamptz;
BEGIN
    SELECT count, window_start INTO v_count, v_start
    FROM public.chat_rate_limit WHERE key = p_key FOR UPDATE;

    IF NOT FOUND OR now() - v_start > make_interval(secs => p_window_secs) THEN
        INSERT INTO public.chat_rate_limit(key, count, window_start) VALUES (p_key, 1, now())
        ON CONFLICT (key) DO UPDATE SET count = 1, window_start = now();
        RETURN true;
    END IF;

    IF v_count >= p_limit THEN
        RETURN false;
    END IF;

    UPDATE public.chat_rate_limit SET count = count + 1 WHERE key = p_key;
    RETURN true;
END; $$;
REVOKE ALL ON FUNCTION public.check_rate_limit(text,int,int) FROM PUBLIC, anon, authenticated;
-- service_role (que usa la edge function) conserva permiso de ejecución.

-- -- SEC-006: escrituras de agotados acotadas al local del staff -----
DROP POLICY IF EXISTS "local_ing_agotados_write" ON public.local_ingredientes_agotados;
CREATE POLICY "local_ing_agotados_write" ON public.local_ingredientes_agotados
    FOR ALL TO authenticated
    USING (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id))
    WITH CHECK (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id));

DROP POLICY IF EXISTS "local_prod_agotados_write" ON public.local_productos_agotados;
CREATE POLICY "local_prod_agotados_write" ON public.local_productos_agotados
    FOR ALL TO authenticated
    USING (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id))
    WITH CHECK (public.has_role_for_local(auth.uid(),'caja',local_id) OR public.has_role_for_local(auth.uid(),'cocina',local_id));

-- -- SEC-008: franchisees -> [] en vez de "permission denied for function ..." --
-- La función devuelve NULL para anon (sin sesión), así que la policy no expone datos.
GRANT EXECUTE ON FUNCTION public.get_user_franchisee_id(uuid) TO anon;

/* ===== */
/* FICHERO: montadito-magic-flow/supabase/migrations/20260623200000_vista_pedidos_agregados.sql */
/* ===== */
--
-- FIX: el Histórico de caja consulta la vista public.v_pedidos_agregados que
-- no existía (error PGRST205 "Could not find the table ... in the schema cache").
-- Esta vista agrega los pedidos por hora/día/mes/año (en horario Europe/Madrid)
-- con nº de tickets e ingresos, que es justo lo que pide Historico.tsx.
--
CREATE OR REPLACE VIEW public.v_pedidos_agregados AS
SELECT
    date_trunc('hour', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS hora,
    date_trunc('day', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS dia,
    date_trunc('month', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS mes,
    date_trunc('year', created_at AT TIME ZONE 'Europe/Madrid') AT TIME ZONE 'Europe/Madrid' AS anio,
    estado,
    count(*)::int AS tickets,
    coalesce(sum(total), 0)::numeric AS ingresos
FROM public.pedidos
GROUP BY 1, 2, 3, 4, 5;

-- Solo el personal autenticado (dashboard) puede leer los agregados.
GRANT SELECT ON public.v_pedidos_agregados TO authenticated;

```